Lecture 2: PixelRNN/CNN, VQ-VAE, Normalizing Flow

자기회귀 모델, 이산 잠재 표현, 그리고 가역 변환 기반 생성 모델

목차

1. 개요
2. PixelRNN / PixelCNN
    - 2.1 핵심 아이디어
    - 2.2 이미지 라이클리후드의 자기회귀적 분해
    - 2.3 컬러 이미지(RGB)에서의 픽셀 생성
    - 2.4 훈련 방법
    - 2.5 Row LSTM
    - 2.6 Diagonal BiLSTM
    - 2.7 PixelCNN
    - 2.8 실험 결과 및 한계
    - 2.9 PyTorch 구현
3. VQ-VAE (Vector Quantized VAE)
    - 3.1 핵심 아이디어
    - 3.2 코드북(Codebook)의 개념
    - 3.3 구조
    - 3.4 양자화(Quantization) 과정
    - 3.5 손실 함수
    - 3.6 Two-Stage 훈련: 생성 모델로 만들기
    - 3.7 VQ-VAE vs VAE 비교
    - 3.8 실용적 의의
    - 3.9 PyTorch 구현
4. Normalizing Flow
    - 4.1 핵심 아이디어
    - 4.2 Change of Variables Theorem
    - 4.3 Log-Likelihood 계산
    - 4.4 합성 함수로의 확장
    - 4.5 Affine Coupling Layer
    - 4.6 자코비안 계산
    - 4.7 교대 분할 (Alternating Pattern)
    - 4.8 전체 학습 및 생성 파이프라인
    - 4.9 VAE와의 비교
    - 4.10 Normalizing Flow → Flow Matching으로의 발전
    - 4.11 PyTorch 구현

5. 전체 요약: 세 모델의 비교
6. 참고문헌

---

1. 개요

이번 강의에서는 세 가지 서로 다른 접근 방식의 생성 모델을 다룹니다:

1.1 핵심 개념

모델	핵심 아이디어	주요 특징
PixelRNN/CNN	자기회귀적(autoregressive) 생성	픽셀을 하나씩 순차적으로 생성
VQ-VAE	이산(discrete) 잠재 표현 학습	코드북 기반의 양자화된 표현
Normalizing Flow	가역(invertible) 변환	정확한 log-likelihood 계산

1.2 선수 지식

- **확률론**: 조건부 확률, chain rule, change of variables
- **선형대수**: 자코비안 행렬, 행렬식(determinant), 역행렬
- **딥러닝 기초**: CNN, RNN/LSTM, Autoencoder
- **VAE**: ELBO, KL divergence, reparameterization trick (Lecture 1 참조)

1.3 학습 목표

1. 자기회귀 모델의 수학적 원리와 masked convolution 이해
2. VQ-VAE의 이산 표현 학습과 straight-through estimator 이해
3. Normalizing Flow의 change of variables와 invertible architecture 이해
4. 세 모델의 장단점 비교 및 실용적 응용 사례 파악

---

2. PixelRNN / PixelCNN

2.1 핵심 아이디어

이미지를 **자기회귀적(autoregressive)**으로 생성한다. 즉, 이전에 생성한 픽셀들을 조건으로 다음 픽셀을 하나씩 예측하는 방식이다.

> 📄 **논문**: van den Oord et al., "Pixel Recurrent Neural Networks", ICML 2016 저자: Aaron van den Oord, Nal Kalchbrenner, Koray Kavukcuoglu (Google DeepMind)

하나의 논문에 세 가지 모델이 포함되어 있다:

1. **Row LSTM**: 행 단위로 LSTM 적용
2. **Diagonal BiLSTM**: 대각선 방향으로 양방향 LSTM 적용
3. **PixelCNN**: Masked convolution으로 전체 이미지 병렬 처리

2.2 이미지 라이클리후드의 자기회귀적 분해

이미지 x는 $n \times n$ 픽셀로 구성되며 총 n^2개의 픽셀이 있다. 이 이미지의 확률을 조건부 확률의 곱으로 분해한다:

$$p(x) = \prod_{i=1}^{n^2} p(x_i \mid x_1, x_2, \ldots, x_{i-1})$$

이것이 어떻게 성립하는지 풀어보면, **chain rule of probability**에 의해:

$$p(x_1, x_2, \ldots, x_{n^2}) = p(x_1) \cdot p(x_2 \mid x_1) \cdot p(x_3 \mid x_1, x_2) \cdots p(x_{n^2} \mid x_1, \ldots, x_{n^2-1})$$

$$= \prod_{i=1}^{n^2} p(x_i \mid x_{<i})$$

여기서 $x_{<i} = \{x_1, x_2, \ldots, x_{i-1}\}$이다.

> ⚠️ **i.i.d. 가정이 아니다!** 각 픽셀은 이전 픽셀들에 의존한다. 곱으로 표현되었지만, 각 항이 조건부 확률이므로 상호 의존성이 반영되어 있다.

수학적 정당성

확률의 chain rule:

$$p(x_1, x_2, \ldots, x_n) = p(x_1) \prod_{i=2}^{n} p(x_i \mid x_1, \ldots, x_{i-1})$$

이것은 **정확한 등식**이며 어떤 근사도 포함하지 않는다. 따라서 PixelRNN/CNN은 **exact likelihood model**이다.

2.3 컬러 이미지(RGB)에서의 픽셀 생성

하나의 픽셀 x_i는 RGB 세 채널로 구성된다. 이를 순차적으로 생성한다:

$$p(x_i \mid x_{<i}) = p(x_{i,R} \mid x_{<i}) \cdot p(x_{i,G} \mid x_{<i}, x_{i,R}) \cdot p(x_{i,B} \mid x_{<i}, x_{i,R}, x_{i,G})$$

풀이 과정

이것은 다시 chain rule을 RGB에 적용한 것이다:

$$p(R, G, B \mid x_{<i}) = p(R \mid x_{<i}) \cdot p(G \mid x_{<i}, R) \cdot p(B \mid x_{<i}, R, G)$$

각 채널의 값은 $\{0, 1, 2, \ldots, 255\}$ 중 하나이므로, **256차원 소프트맥스 분포**를 출력한다.

예를 들어, Red 채널의 확률 분포가 다음과 같이 나왔다면:

$$p(x_{i,R} = k \mid x_{<i}) \quad \text{for } k = 0, 1, \ldots, 255$$

가장 확률이 높은 값(예: $k = 72$)을 선택하거나, **확률적으로 샘플링**하여 다양한 이미지를 생성한다.

왜 256-way classification인가?

- Regression (회귀)로 픽셀 값을 예측하면: 흐릿한(blurry) 결과
- Classification (분류)로 픽셀 값을 예측하면: 선명한 결과

각 픽셀 값에 대한 확률 분포를 모델링하므로, **multimodal distribution**을 표현할 수 있다.

2.4 훈련 방법

- **Teacher forcing**: 훈련 시에는 모델이 생성한 값 대신 **정답 픽셀값**을 다음 스텝의 입력으로 사용
- **Loss**: 각 픽셀에서 256차원 cross-entropy loss 계산

$$\mathcal{L} = -\sum_{i=1}^{n^2} \sum_{c \in \{R,G,B\}} \sum_{k=0}^{255} 1[x_{i,c} = k] \log \hat{p}(x_{i,c} = k \mid x_{<i})$$

여기서 $1[\cdot]$는 indicator function (원핫 인코딩과 동일).

Teacher Forcing의 중요성

- **훈련**: 정답 픽셀을 입력 → 빠른 학습, 안정적 그래디언트
- **생성**: 모델 출력을 입력 → 순차적 생성

훈련과 생성 간의 이러한 불일치를 **exposure bias**라고 하며, 자기회귀 모델의 고질적인 문제이다.

2.5 Row LSTM

구조

일반 RNN 대신 **Conv-LSTM**을 사용한다. 이미지를 다루므로 행렬곱 대신 **컨볼루션 연산**을 수행한다.

Dependency 구조

현재 생성할 픽셀의 위치를 기준으로:

- **입력** x: 현재 행의 자신 포함 왼쪽 3개 픽셀
- **이전 hidden state** h_{t-1}: 바로 윗행의 대각선 위 3개 픽셀 (역삼각형 형태)

이 두 입력을 LSTM 연산에 넣어 현재 픽셀의 hidden state h_t를 계산한다.

LSTM 수식

Conv-LSTM의 게이트 연산:

$$i_t = \sigma(W_{xi} * x_t + W_{hi} * h_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf} * x_t + W_{hf} * h_{t-1} + b_f)$$

$$o_t = \sigma(W_{xo} * x_t + W_{ho} * h_{t-1} + b_o)$$

$$g_t = \tanh(W_{xg} * x_t + W_{hg} * h_{t-1} + b_g)$$

Cell state와 hidden state 업데이트:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh(c_t)$$

여기서 $*$는 convolution, $\odot$는 element-wise multiplication이다.

병렬 처리

- **훈련 시**: 정답이 모두 있으므로, **행(row) 단위로** 한꺼번에 계산 가능
- **생성 시**: 하나씩 순차적으로 생성해야 함 (느림)

한계

Receptive field가 **위쪽 삼각형 영역**으로 제한된다. 같은 행의 왼쪽 픽셀이나 아래쪽 영역을 직접 참조하지 못한다.

2.6 Diagonal BiLSTM

개선점

Row LSTM의 제한적 receptive field를 극복한다.

Dependency 구조

현재 생성할 픽셀(예: 8번)을 기준으로:

- **입력** x: 자신 포함 왼쪽 3개 픽셀 (Row LSTM과 동일)
- **이전 hidden state** h_{t-1}: **수직 위 1개 + 왼쪽 위 1개** (총 2개)

$$h_t = f(x_{\text{context}}, h_{\text{above}}, h_{\text{left-above}})$$

대각선 방향 생성

대각선 방향으로 한 번에 생성한다:

```
1단계: (1,1)
2단계: (1,2), (2,1)  ← 동시 계산 가능
3단계: (1,3), (2,2), (3,1)  ← 동시 계산 가능
...
```

각 대각선 내의 픽셀들은 서로 독립적이므로 **병렬 계산**이 가능하다.

양방향(Bi-directional)

- 왼쪽 위 → 오른쪽 아래 방향으로 한번
- 오른쪽 아래 → 왼쪽 위 방향으로 한번
- 두 결과의 평균 또는 concatenation으로 최종 픽셀 값 결정

$$h_t = [h_t^{\text{forward}}, h_t^{\text{backward}}]$$

왜 전체를 참조하게 되는가

9번 픽셀을 만들 때 5번, 6번을 참조하고, 5번을 만들 때 1번, 2번을 참조하고... 이런 식으로 **간접적으로 모든 이전 픽셀을 참조**하게 된다.

이것을 **transitive closure**라고 하며, 충분한 층을 쌓으면 전체 이미지의 정보가 전달된다.

구현 트릭

대각선 형태를 컨볼루션하기 편하도록 **아래줄을 오른쪽으로 한 칸 시프트**하여 직사각형으로 만든 뒤 연산한다.

2.7 PixelCNN

핵심: 마스크드 컨볼루션

특수한 **마스크 필터**를 사용하여 이미지 전체를 한번에 처리한다.

```
마스크 예시 (7×7 필터):

1 1 1 1 1 1 1
1 1 1 1 1 1 1
1 1 1 1 1 1 1
1 1 1 ★ 0 0 0    ← ★ = 현재 예측 위치
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
```

- 마스크가 1인 위치: 이미 생성된 픽셀 → 참조 가능
- 마스크가 0인 위치: 아직 생성되지 않은 픽셀 → 참조 불가
- ★ 위치: 현재 생성할 픽셀

Masked Convolution의 두 가지 타입

Type A: 첫 번째 층에서 사용

- 중심 픽셀 자신을 **포함하지 않음**
- RGB 순서에서 R을 예측할 때 R 자신은 사용 불가

Type B: 이후 층에서 사용

- 중심 픽셀 자신을 **포함**
- 이미 이전 층에서 계산된 값이므로 사용 가능

장점

	Row LSTM	Diagonal BiLSTM	PixelCNN
훈련 병렬성	행 단위	대각선 단위	**이미지 전체**
주변 참조	위쪽 삼각형	전체(간접)	**자기 주변 직접**
성능	보통	좋음	**가장 좋음**
훈련 속도	느림	보통	**빠름**

- **훈련 시**: 마스크 컨볼루션으로 이미지 전체를 한번에 학습 (모든 정답이 있으므로)
- **생성 시**: 여전히 하나씩 순차 생성 (정답이 없으므로)

Blind Spot 문제

초기 PixelCNN에는 **blind spot** 문제가 있었다:

- 마스크 구조상 특정 영역을 참조하지 못함
- **Gated PixelCNN** (2016)에서 vertical stack과 horizontal stack으로 해결

2.8 실험 결과 및 한계

성능

- **CIFAR-10** (32×32): NLL = 3.03 bits/dim
- **ImageNet** (64×64): NLL = 3.86 bits/dim

시각적 품질

- CIFAR-10(32×32)에서 무언가를 만들어내지만, 뚜렷하지 않음
- 이미지 생성에서는 **GAN에 비해 성능 열세**
- 생성 속도가 매우 느림 (32×32 이미지 하나에 수 초)

성공 사례: 오디오 분야

- **WaveNet** (2016): 음성 합성의 보코더로 혁신
- **SampleRNN** (2017): 음악 생성
- **Jukebox** (2020): OpenAI의 음악 생성 모델

오디오는 1D 시퀀스이므로 자기회귀 모델이 더 적합하다.

2.9 PyTorch 구현

Masked Convolution

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MaskedConv2d(nn.Conv2d):
    """
    Masked 2D Convolution for PixelCNN

    Args:
        mask_type: 'A' for first layer, 'B' for subsequent layers
    """
    def __init__(self, mask_type, *args, **kwargs):
        super(MaskedConv2d, self).__init__(*args, **kwargs)
        assert mask_type in ['A', 'B']
        self.register_buffer('mask', torch.zeros_like(self.weight.da

        _, _, kH, kW = self.weight.size()
        # Center position
        center_h, center_w = kH // 2, kW // 2

        # Create mask
        self.mask[:, :, :center_h, :] = 1  # All rows above center
        self.mask[:, :, center_h, :center_w] = 1  # Left side of cen

        if mask_type == 'B':
            self.mask[:, :, center_h, center_w] = 1  # Include cente

    def forward(self, x):
        self.weight.data *= self.mask
        return super(MaskedConv2d, self).forward(x)


class GatedActivation(nn.Module):
    """
    Gated activation unit: tanh(W_f * x) ⊙ sigmoid(W_g * x)
    """
    def __init__(self, in_channels, out_channels):
        super(GatedActivation, self).__init__()
        self.conv = nn.Conv2d(in_channels, 2 * out_channels, kernel_

    def forward(self, x):
        x = self.conv(x)
        tanh_out, sigmoid_out = x.chunk(2, dim=1)
        return torch.tanh(tanh_out) * torch.sigmoid(sigmoid_out)


class ResidualBlock(nn.Module):
    """
    Residual block for PixelCNN
    """
    def __init__(self, n_filters):
        super(ResidualBlock, self).__init__()
```

```
        self.conv1 = MaskedConv2d('B', n_filters, n_filters,
                                  kernel_size=3, padding=1)
        self.gated = GatedActivation(n_filters, n_filters)
        self.conv2 = nn.Conv2d(n_filters, n_filters, kernel_size=1)

    def forward(self, x):
        residual = x
        x = self.conv1(x)
        x = self.gated(x)
        x = self.conv2(x)
        return x + residual


class PixelCNN(nn.Module):
    """
    PixelCNN for image generation

    Args:
        input_channels: Number of input channels (3 for RGB)
        n_filters: Number of filters in hidden layers
        n_layers: Number of residual blocks
        n_classes: Number of output classes per channel (256 for 8-b
    """
    def __init__(self, input_channels=3, n_filters=64, n_layers=12,
        super(PixelCNN, self).__init__()
        self.n_classes = n_classes

        # First layer: Masked Conv Type A
        self.first_layer = MaskedConv2d('A', input_channels, n_filte
                                        kernel_size=7, padding=3)

        # Residual blocks
        self.residual_blocks = nn.ModuleList([
            ResidualBlock(n_filters) for _ in range(n_layers)
        ])

        # Output layers
        self.out1 = nn.Conv2d(n_filters, n_filters, kernel_size=1)
        self.out2 = nn.Conv2d(n_filters, input_channels * n_classes,

    def forward(self, x):
        """
        Args:
            x: Input tensor of shape (B, C, H, W)

        Returns:
            logits: Output logits of shape (B, C*n_classes, H, W)
        """
        # First layer
        x = self.first_layer(x)
        x = F.relu(x)
```

```python
        # Residual blocks
        for block in self.residual_blocks:
            x = block(x)

        # Output layers
        x = F.relu(self.out1(x))
        logits = self.out2(x)

        return logits

    def sample(self, n_samples, img_shape=(3, 32, 32), device='cuda
        """
        Generate samples from the model

        Args:
            n_samples: Number of samples to generate
            img_shape: Shape of generated images (C, H, W)
            device: Device to generate on

        Returns:
            samples: Generated images of shape (n_samples, C, H, W)
        """
        self.eval()
        C, H, W = img_shape
        samples = torch.zeros(n_samples, C, H, W, dtype=torch.long)

        with torch.no_grad():
            # Generate pixel by pixel
            for h in range(H):
                for w in range(W):
                    for c in range(C):
                        # Convert to float for model input
                        x = samples.float() / (self.n_classes - 1)

                        # Forward pass
                        logits = self.forward(x)

                        # Extract logits for current pixel and chan
                        # logits shape: (B, C*n_classes, H, W)
                        pixel_logits = logits[:, c*self.n_classes:(

                        # Sample from categorical distribution
                        probs = F.softmax(pixel_logits, dim=1)
                        pixel_sample = torch.multinomial(probs, 1).

                        # Update samples
                        samples[:, c, h, w] = pixel_sample

        # Convert to [0, 1] range
        return samples.float() / (self.n_classes - 1)
```

```python
# Training function
def train_pixelcnn(model, train_loader, optimizer, device, n_classes
    """
    Train PixelCNN for one epoch

    Args:
        model: PixelCNN model
        train_loader: DataLoader for training data
        optimizer: Optimizer
        device: Device to train on
        n_classes: Number of output classes

    Returns:
        avg_loss: Average loss for the epoch
    """
    model.train()
    total_loss = 0

    for batch_idx, (data, _) in enumerate(train_loader):
        # Assuming data is in [0, 1] range, convert to [0, n_classes
        data = (data * (n_classes - 1)).long().to(device)

        # Forward pass
        data_float = data.float() / (n_classes - 1)
        logits = model(data_float)

        # Reshape for cross-entropy loss
        # logits: (B, C*n_classes, H, W) -> (B, n_classes, C, H, W)
        B, _, H, W = logits.shape
        C = data.shape[1]
        logits = logits.view(B, C, n_classes, H, W).permute(0, 2, 1

        # Compute loss
        loss = F.cross_entropy(logits, data)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    return total_loss / len(train_loader)


# Example usage
if __name__ == "__main__":
    # Hyperparameters
    batch_size = 32
    n_epochs = 100
    learning_rate = 1e-3
```

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'

# Initialize model
model = PixelCNN(input_channels=3, n_filters=128, n_layers=15, 
model = model.to(device)

# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rat

# Training loop (assuming train_loader is defined)
# for epoch in range(n_epochs):
#     loss = train_pixelcnn(model, train_loader, optimizer, dev
#     print(f"Epoch {epoch+1}, Loss: {loss:.4f}")

# # Generate samples
# samples = model.sample(n_samples=16, img_shape=(3, 32, 32), de
```

주요 하이퍼파라미터

파라미터	권장 값	설명
n_filters	128-256	Hidden layer 필터 수
n_layers	12-20	Residual block 수
kernel_size	3, 5, 7	첫 층은 7×7, 나머지는 3×3
learning_rate	1e-3 to 1e-4	Adam optimizer
batch_size	32-128	GPU 메모리에 따라 조정

---

3. VQ-VAE (Vector Quantized VAE)

3.1 핵심 아이디어

VAE의 **이산화(discrete) 버전**이다.

	VAE	VQ-VAE
잠재 공간	연속(continuous)	이산(discrete)
Prior	표준 정규분포	코드북(codebook)의 유한 원소
비유	아날로그	디지털

> 📄 **논문**: van den Oord et al., "Neural Discrete Representation Learning", NeurIPS 2017 저자: Aaron van den Oord, Oriol Vinyals, Koray Kavukcuoglu (Google DeepMind)

3.2 코드북(Codebook)의 개념

코드북: K개의 D차원 벡터로 구성된 딕셔너리

$$\mathbf{e} = \{e_1, e_2, \ldots, e_K\}, \quad e_k \in \mathbb{R}^D$$

자연어 처리에서의 **vocabulary**와 동일한 역할:

- 자연어: 정해진 단어(토큰)로만 문장을 표현
- VQ-VAE: 정해진 코드 벡터로만 이미지를 표현

왜 이산 표현인가?

1. **Posterior collapse 방지**: VAE의 고질적 문제 해결
2. **트랜스포머 호환**: 이산 토큰은 트랜스포머의 자연스러운 입력
3. **압축 효율**: 연속 벡터보다 저장/전송 효율적
4. **해석 가능성**: 각 코드가 의미 있는 시각적 패턴을 학습

3.3 구조

```
입력 이미지 (H×W×3)
    ↓
[CNN Encoder]
    ↓
연속 feature map (h×w×D)  →  코드북 {e_1,...,e_K}에서 최근접 매칭
    ↓
양자화된 표현 (h×w, 인덱스)  →  코드 벡터로 치환 (h×w×D)
    ↓
[CNN Decoder (Transposed Conv)]
    ↓
복원 이미지 (H×W×3)
```

3.4 양자화(Quantization) 과정

인코더의 출력 feature map에서 각 공간 위치 (i, j)의 벡터 $z_e(x)_{ij} \in \mathbb{R}^D$에 대해:

$$q(z_{ij} = k \mid x) = \begin{cases} 1 & \text{if } k = \arg\min_j \|z_e(x)_{ij} - e_j\|_2 \\ 0 & \text{otherwise} \end{cases}$$

즉, 코드북의 모든 벡터와의 **L2 거리를 계산**하여 가장 가까운 코드로 치환한다:

$$z_q(x)_{ij} = e_k, \quad \text{where } k = \arg\min_j \|z_e(x)_{ij} - e_j\|_2$$

구체적 수치 예시

논문의 설정:

- 입력: $256 \times 256 \times 3$
- 인코더 출력: $32 \times 32 \times 64$ (필터 64개)

- 코드북: $K = 512$개, 각 $D = 64$차원

각 공간 위치 (i, j)의 64차원 벡터를 512개 코드와 비교하여 가장 가까운 것으로 교체한다.

결과: $32 \times 32 \times 64 \rightarrow 32 \times 32 \times 1$ (인덱스만 저장)

이것을 **코드 인덱스 맵(code index map)**이라 부른다. 원본 $256 \times 256 \times 3$에서 $32 \times 32 \times 1$로의 대폭적인 압축이다.

압축률 계산

- 원본: $256 \times 256 \times 3 \times 8 = 1,572,864$ bits
- 압축: $32 \times 32 \times \log_2(512) = 9,216$ bits
- 압축률: **약 170배**

3.5 손실 함수

VQ-VAE의 손실 함수는 세 항으로 구성된다:

$$\mathcal{L} = \underbrace{\|x - \hat{x}\|_2^2}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook Loss}} + \underbrace{\beta\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment Loss}}$$

여기서 $\text{sg}[\cdot]$은 **stop-gradient** 연산자이다.

각 항의 의미와 그래디언트 흐름

(1) Reconstruction Loss: $\|x - \hat{x}\|_2^2$

- **목적**: 입력과 복원의 유사도 최대화
- **그래디언트 흐름 문제**: 양자화 단계에서 그래디언트가 끊긴다 (이산 연산이므로)
- **해결**: **Straight-Through Estimator** — 디코더에서 계산된 그래디언트를 그대로 복사하여 인코더로 전달

```
순전파: z_e → [양자화: argmin] → z_q → Decoder → x̂
역전파: z_e ← [그래디언트 복사] ← z_q ← Decoder ← Loss
```

수식으로 표현하면:

$$\frac{\partial \mathcal{L}}{\partial z_e} = \frac{\partial \mathcal{L}}{\partial z_q}$$

(2) Codebook Loss: $\|\text{sg}[z_e(x)] - e\|_2^2$

- **목적**: 코드북 벡터 e를 인코더 출력 $z_e(x)$에 가까이 이동
- $\text{sg}[z_e(x)]$: 인코더 출력의 그래디언트를 **차단** → 인코더는 업데이트되지 않음
- **결과**: 이 항으로는 **코드북만 업데이트**됨

수식적으로 풀어보면:

$$\frac{\partial}{\partial e}\|\text{sg}[z_e] - e\|_2^2 = -2(\text{sg}[z_e] - e) = 2(e - z_e)$$

$$\frac{\partial}{\partial z_e}\|\text{sg}[z_e] - e\|_2^2 = 0 \quad (\text{stop-gradient})$$

따라서 코드북 업데이트:

$$e \leftarrow e - \eta \cdot 2(e - z_e) = e + 2\eta(z_e - e)$$

즉, 코드북 벡터가 인코더 출력 쪽으로 이동한다.

(3) Commitment Loss: $\beta\|z_e(x) - \text{sg}[e]\|_2^2$

- **목적**: 인코더 출력 $z_e(x)$를 코드북 벡터 e에 가까이 이동
- $\text{sg}[e]$: 코드북의 그래디언트를 **차단** → 코드북은 업데이트되지 않음
- **결과**: 이 항으로는 **인코더만 업데이트**됨

수식적으로 풀어보면:

$$\frac{\partial}{\partial z_e}\beta\|z_e - \text{sg}[e]\|_2^2 = 2\beta(z_e - \text{sg}[e]) = 2\beta(z_e - e)$$

$$\frac{\partial}{\partial e}\beta\|z_e - \text{sg}[e]\|_2^2 = 0 \quad (\text{stop-gradient})$$

따라서 인코더 업데이트:

$$z_e \leftarrow z_e - \eta \cdot 2\beta(z_e - e)$$

즉, 인코더 출력이 코드북 벡터 쪽으로 이동한다.

왜 이렇게 분리하는가?

그래디언트를 동시에 양쪽으로 흘리면:

- 코드북 값이 매 스텝마다 급격히 변화
- 코드로서의 안정적 역할을 못 함
- "Moving target" 문제 발생

따라서 **교대로** 업데이트하여 안정적 학습을 보장한다.

학습 대상 정리

손실 항	업데이트 대상	차단 대상	그래디언트
Reconstruction Loss	인코더 + 디코더	(straight-through)	$\nabla_{z_e}\|\|x - \hat{x}\|\|^2$
Codebook Loss	코드북	인코더 (sg)	$\nabla_e\|\|z_e - e\|\|^2$

Commitment Loss	인코더	코드북 (sg)	$\nabla_{z_e}\|\|z_e - e\|\|^2$

Hyperparameter β

- 논문에서는 $\beta = 0.25$ 사용
- β가 너무 크면: 인코더가 코드북에 과도하게 의존, 표현력 감소
- β가 너무 작으면: 인코더가 코드북을 무시, 양자화 실패

3.6 Two-Stage 훈련: 생성 모델로 만들기

VQ-VAE 자체는 오토인코더일 뿐, 생성 모델이 아니다. 생성 모델로 사용하려면 **두 단계**가 필요하다.

Stage 1: VQ-VAE 훈련 + 코드 인덱스 맵 추출

1. 인코더, 디코더, 코드북을 위의 손실 함수로 훈련
2. 훈련 완료 후, 모든 훈련 이미지를 인코더에 통과시켜 **코드 인덱스 맵** 추출
3. 예: ImageNet 2100만 장 → 32×32 코드 인덱스 맵 2100만 개 저장

Stage 2: PixelCNN Prior 훈련

코드 인덱스 맵을 "이미지"로 간주하고, **PixelCNN**으로 자기회귀적 생성을 학습한다.

$$p(z) = \prod_{i=1}^{h \times w} p(z_i \mid z_1, \ldots, z_{i-1})$$

각 z_i는 K개 코드 중 하나이므로, K**-way 분류 문제**가 된다.

생성 시

```
PixelCNN Prior로 코드 인덱스 맵 생성 (32×32)
    ↓
코드북에서 벡터 조회 → 양자화된 표현 (32×32×64)
    ↓
[VQ-VAE Decoder]
    ↓
생성 이미지 (256×256×3)
```

왜 랜덤하게 채우면 안 되는가?

- **유니폼하게 샘플링**하면: 아무런 구조 없는 노이즈 출력
  - $p(z_i) = \text{Uniform}(\{1, 2, \ldots, K\})$
  - 각 코드가 독립적으로 선택됨
  - 코드 간 상관관계 없음 → 의미 없는 이미지

- **PixelCNN Prior**로 채우면: 데이터 분포를 학습한 의미 있는 코드 배치 → 그럴싸한 이미지
    - $p(z) = \prod_{i=1}^{h \times w} p(z_i \mid z_{<i})$
    - 코드 간 의존성 학습
    - 의미 있는 구조 생성

이는 VAE에서 랜덤 벡터 vs 표준 정규분포 샘플링의 차이와 동일한 논리이다.

조합의 다양성

가능한 이미지 수: $K^{h \times w} = 512^{32 \times 32} = 512^{1024}$ — 사실상 **무한대**

하지만 PixelCNN이 학습한 분포 $p(z)$는 이 거대한 공간의 극히 일부만을 커버한다.

3.7 VQ-VAE vs VAE 비교

	VAE	**VQ-VAE**
잠재 표현	연속 벡터 $z \in \mathbb{R}^d$	이산 코드 인덱스 $z \in \{1, \ldots, K\}^{h \times w}$
Prior	표준 정규분포 $\mathcal{N}(0, I)$	PixelCNN으로 학습
생성 품질	Blurry (흐릿함)	**선명**
Posterior Collapse	있음 (KL → 0 문제)	**없음**
트랜스포머 호환성	낮음 (연속 벡터)	**높음** (이산 토큰 기반)
Exact likelihood	ELBO (하한)	**Exact** (PixelCNN prior 사용 시)
훈련 단계	1단계 (end-to-end)	2단계 (VQ-VAE + Prior)

Posterior Collapse가 없는 이유

VAE에서 posterior collapse:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x) \| p(z))$$

디코더가 너무 강력하면 KL 항이 0으로 수렴 → z가 무시됨

VQ-VAE에서는:

- 양자화가 **강제**로 정보 병목 생성
- 코드북 크기 K로 표현력 제한
- z를 무시하면 복원 불가능

3.8 실용적 의의

직접적 응용

- **DALL-E** (2021): VQ-VAE의 변형인 dVAE 사용
- **VQ-GAN** (2021): GAN loss 추가로 품질 향상
- **MaskGIT** (2022): Masked transformer로 prior 학습

간접적 영향

- 이미지 생성 자체보다는 **저차원 이산 표현을 얻는 데** 더 많이 활용
- 저차원 표현을 얻은 후, 디퓨전 모델이나 트랜스포머로 생성에 활용
- **Latent Diffusion Models**: VQ-VAE 대신 VAE 사용하지만, 아이디어는 동일

후속 연구

- **VQ-VAE-2** (2019): 계층적 구조로 품질 대폭 향상
- **Finite Scalar Quantization** (2023): 코드북 없이 양자화
- **Residual VQ**: 여러 코드북을 residual 형태로 사용

3.9 PyTorch 구현

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class VectorQuantizer(nn.Module):
    """
    Vector Quantization layer for VQ-VAE

    Args:
        num_embeddings: Number of codebook vectors (K)
        embedding_dim: Dimension of each codebook vector (D)
        commitment_cost: Weight for commitment loss (beta)
    """
    def __init__(self, num_embeddings, embedding_dim, commitment_cos
        super(VectorQuantizer, self).__init__()

        self.embedding_dim = embedding_dim
        self.num_embeddings = num_embeddings
        self.commitment_cost = commitment_cost

        # Initialize codebook
        self.embedding = nn.Embedding(num_embeddings, embedding_dim)
        self.embedding.weight.data.uniform_(-1/num_embeddings, 1/nu

    def forward(self, inputs):
        """
        Args:
            inputs: Encoder outputs of shape (B, D, H, W)

        Returns:
            quantized: Quantized vectors of shape (B, D, H, W)
            loss: VQ loss (codebook loss + commitment loss)
            perplexity: Measure of codebook usage
        """
        # Convert inputs from (B, D, H, W) to (B, H, W, D)
        inputs = inputs.permute(0, 2, 3, 1).contiguous()
        input_shape = inputs.shape

        # Flatten input: (B, H, W, D) -> (B*H*W, D)
        flat_input = inputs.view(-1, self.embedding_dim)

        # Calculate distances to codebook vectors
        # (B*H*W, D) - (K, D) -> (B*H*W, K)
        distances = (torch.sum(flat_input**2, dim=1, keepdim=True)
                    + torch.sum(self.embedding.weight**2, dim=1)
                    - 2 * torch.matmul(flat_input, self.embedding.we

        # Encoding: find nearest codebook vector
        encoding_indices = torch.argmin(distances, dim=1).unsqueeze
        encodings = torch.zeros(encoding_indices.shape[0], self.num_
                                device=inputs.device)
        encodings.scatter_(1, encoding_indices, 1)
```

```
        # Quantize: lookup codebook vectors
        quantized = torch.matmul(encodings, self.embedding.weight).v

        # Loss
        e_latent_loss = F.mse_loss(quantized.detach(), inputs)  # Co
        q_latent_loss = F.mse_loss(quantized, inputs.detach())  # Co
        loss = q_latent_loss + self.commitment_cost * e_latent_loss

        # Straight-through estimator
        quantized = inputs + (quantized - inputs).detach()

        # Perplexity: measure of codebook usage
        avg_probs = torch.mean(encodings, dim=0)
        perplexity = torch.exp(-torch.sum(avg_probs * torch.log(avg_

        # Convert quantized from (B, H, W, D) to (B, D, H, W)
        quantized = quantized.permute(0, 3, 1, 2).contiguous()

        return quantized, loss, perplexity


class ResidualBlock(nn.Module):
    """Residual block with two Conv layers"""
    def __init__(self, in_channels, num_hiddens, num_residual_hidder
        super(ResidualBlock, self).__init__()
        self.block = nn.Sequential(
            nn.ReLU(True),
            nn.Conv2d(in_channels, num_residual_hiddens, 3, padding=
            nn.ReLU(True),
            nn.Conv2d(num_residual_hiddens, num_hiddens, 1, bias=Fa
        )

    def forward(self, x):
        return x + self.block(x)


class Encoder(nn.Module):
    """VQ-VAE Encoder"""
    def __init__(self, in_channels, num_hiddens, num_residual_layers
        super(Encoder, self).__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, num_hiddens // 2, 4, stride=2, pa
            nn.ReLU(True),
            nn.Conv2d(num_hiddens // 2, num_hiddens, 4, stride=2, pa
            nn.ReLU(True),
            nn.Conv2d(num_hiddens, num_hiddens, 3, padding=1),
        )

        self.residual_stack = nn.Sequential(
            *[ResidualBlock(num_hiddens, num_hiddens, num_residual_
```

```
            *[ResidualBlock(num_hiddens, num_hiddens, num_residual_l
              for _ in range(num_residual_layers)]
        )

    def forward(self, x):
        x = self.conv(x)
        return self.residual_stack(x)


class Decoder(nn.Module):
    """VQ-VAE Decoder"""
    def __init__(self, in_channels, num_hiddens, num_residual_layers
        super(Decoder, self).__init__()

        self.conv1 = nn.Conv2d(in_channels, num_hiddens, 3, padding=

        self.residual_stack = nn.Sequential(
            *[ResidualBlock(num_hiddens, num_hiddens, num_residual_l
              for _ in range(num_residual_layers)]
        )

        self.conv_trans = nn.Sequential(
            nn.ConvTranspose2d(num_hiddens, num_hiddens // 2, 4, str
            nn.ReLU(True),
            nn.ConvTranspose2d(num_hiddens // 2, 3, 4, stride=2, pad
        )

    def forward(self, x):
        x = self.conv1(x)
        x = self.residual_stack(x)
        return self.conv_trans(x)


class VQVAE(nn.Module):
    """
    Complete VQ-VAE model

    Args:
        num_hiddens: Number of hidden channels
        num_residual_layers: Number of residual blocks
        num_residual_hiddens: Hidden channels in residual blocks
        num_embeddings: Size of codebook (K)
        embedding_dim: Dimension of codebook vectors (D)
        commitment_cost: Weight for commitment loss (beta)
    """
    def __init__(self, num_hiddens=128, num_residual_layers=2, num_r
                 num_embeddings=512, embedding_dim=64, commitment_co
        super(VQVAE, self).__init__()

        self.encoder = Encoder(3, num_hiddens, num_residual_layers,

        self.pre_vq_conv = nn.Conv2d(num_hiddens, embedding_dim, 1)
```

```python
        self.vq = VectorQuantizer(num_embeddings, embedding_dim, cor

        self.decoder = Decoder(embedding_dim, num_hiddens, num_resi(

    def forward(self, x):
        """
        Args:
            x: Input images of shape (B, 3, H, W)

        Returns:
            recon: Reconstructed images
            vq_loss: VQ loss
            perplexity: Codebook usage measure
        """
        z = self.encoder(x)
        z = self.pre_vq_conv(z)
        quantized, vq_loss, perplexity = self.vq(z)
        recon = self.decoder(quantized)

        return recon, vq_loss, perplexity

    def encode(self, x):
        """Encode images to codebook indices"""
        z = self.encoder(x)
        z = self.pre_vq_conv(z)

        # Get quantized representation
        z = z.permute(0, 2, 3, 1).contiguous()
        flat_z = z.view(-1, self.vq.embedding_dim)

        # Find nearest codebook vectors
        distances = (torch.sum(flat_z**2, dim=1, keepdim=True)
                    + torch.sum(self.vq.embedding.weight**2, dim=1)
                    - 2 * torch.matmul(flat_z, self.vq.embedding.we:

        encoding_indices = torch.argmin(distances, dim=1)

        # Reshape to (B, H, W)
        B, H, W, _ = z.shape
        encoding_indices = encoding_indices.view(B, H, W)

        return encoding_indices

    def decode_from_indices(self, indices):
        """Decode from codebook indices"""
        # Lookup codebook vectors
        quantized = self.vq.embedding(indices)

        # Reshape to (B, H, W, D) -> (B, D, H, W)
        quantized = quantized.permute(0, 3, 1, 2).contiguous()
```

```python
        return self.decoder(quantized)


# Training function
def train_vqvae(model, train_loader, optimizer, device, num_epochs=
    """
    Train VQ-VAE

    Args:
        model: VQVAE model
        train_loader: DataLoader for training data
        optimizer: Optimizer
        device: Device to train on
        num_epochs: Number of training epochs
    """
    model.train()

    for epoch in range(num_epochs):
        total_recon_loss = 0
        total_vq_loss = 0
        total_perplexity = 0

        for batch_idx, (data, _) in enumerate(train_loader):
            data = data.to(device)
            optimizer.zero_grad()

            # Forward pass
            recon, vq_loss, perplexity = model(data)

            # Reconstruction loss
            recon_loss = F.mse_loss(recon, data)

            # Total loss
            loss = recon_loss + vq_loss

            # Backward pass
            loss.backward()
            optimizer.step()

            total_recon_loss += recon_loss.item()
            total_vq_loss += vq_loss.item()
            total_perplexity += perplexity.item()

        # Print statistics
        avg_recon_loss = total_recon_loss / len(train_loader)
        avg_vq_loss = total_vq_loss / len(train_loader)
        avg_perplexity = total_perplexity / len(train_loader)

        print(f"Epoch {epoch+1}/{num_epochs}")
        print(f"  Recon Loss: {avg_recon_loss:.4f}")
        print(f"  VQ Loss: {avg_vq_loss:.4f}")
        print(f"  Perplexity: {avg_perplexity:.2f}")
```

```
# Example usage
if __name__ == "__main__":
    # Hyperparameters
    device = 'cuda' if torch.cuda.is_available() else 'cpu'

    # Model
    model = VQVAE(
        num_hiddens=128,
        num_residual_layers=2,
        num_residual_hiddens=32,
        num_embeddings=512,
        embedding_dim=64,
        commitment_cost=0.25
    ).to(device)

    # Optimizer
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    # Training (assuming train_loader is defined)
    # train_vqvae(model, train_loader, optimizer, device, num_epochs

    # Encode images to indices
    # with torch.no_grad():
    #     indices = model.encode(images)  # Shape: (B, H, W)

    # Decode from indices
    # with torch.no_grad():
    #     reconstructed = model.decode_from_indices(indices)
```

주요 하이퍼파라미터

파라미터	권장 값	설명
num_embeddings (K)	128-512	코드북 크기 (더 크면 표현력↑, 메모리↑)
embedding_dim (D)	64-256	코드 벡터 차원
commitment_cost (β)	0.25	Commitment loss 가중치
num_hiddens	128-256	Encoder/Decoder 히든 채널 수
num_residual_layers	2-4	Residual block 수
learning_rate	1e-3 to 1e-4	Adam optimizer

성능 지표

- **Perplexity**: 코드북 사용률 측정
    - Perplexity ≈ K: 모든 코드 균등하게 사용 (이상적)

    - Perplexity ≪ K: 일부 코드만 사용 (코드북 붕괴)
- **Codebook usage**: 실제 사용된 코드의 비율
    - 80% 이상 사용되면 양호

---

4. Normalizing Flow

4.1 핵심 아이디어

역함수가 존재하는(invertible) 뉴럴넷을 사용하여, 하나의 네트워크로 인코딩과 디코딩을 모두 수행한다.

	VAE	Normalizing Flow
네트워크 수	2개 (인코더 + 디코더)	**1개** (invertible)
차원 변화	x와 z의 차원이 다름	x와 z의 차원이 **같음**
목적 함수	ELBO (하한)	**정확한 log-likelihood**
Unknown term	$D_{KL}(q\|\|p) \geq 0$	**없음**

```
Forward:  x → f(x) → z    (x를 단순한 분포 z로 변환)
Inverse:  z → f⁻¹(z) → x  (단순한 분포에서 복잡한 데이터 생성)
```

> 📄 **대표 논문**:
>
> - NICE: Dinh et al., "NICE: Non-linear Independent Components Estimation", ICLR 2015
> - Real NVP: Dinh et al., "Density estimation using Real NVP", ICLR 2017 저자: Laurent Dinh, Jascha Sohl-Dickstein, Samy Bengio

4.2 Change of Variables Theorem

이것이 Normalizing Flow의 수학적 기반이다.

1차원 경우

확률변수 z의 분포 $p_Z(z)$를 알고, $x = g(z)$ (또는 동치로 $z = f(x)$)라는 변환이 존재할 때:

$$p_Z(z)|dz| = p_X(x)|dx|$$

이것은 **확률의 보존**을 의미한다: 변환 전후로 확률의 총량은 보존되어야 한다.

풀이

$p_X(x)$에 대해 풀면:

$$p_X(x) = p_Z(z) \left| \frac{dz}{dx} \right|$$

$z = f(x)$를 대입하면:

$$p_X(x) = p_Z(f(x)) \left| \frac{df(x)}{dx} \right|$$

직관적 이해

z의 범위가 $[-1, 1]$이고 $x = 2z$로 변환한다면:

- x의 범위는 $[-2, 2]$로 2배 넓어짐
- 확률 밀도는 절반으로 줄어야 총 적분이 1 유지
- $\frac{dz}{dx} = \frac{1}{2}$이므로 $p_X(x) = p_Z(z) \cdot \frac{1}{2}$

수치 예시:

- $z \sim \text{Uniform}(-1, 1)$이면 $p_Z(z) = 0.5$
- $x = 2z$이므로 $x \sim \text{Uniform}(-2, 2)$
- $p_X(x) = p_Z(z) \cdot |\frac{dz}{dx}| = 0.5 \cdot 0.5 = 0.25$
- 검증: $\int_{-2}^{2} 0.25 dx = 1$ ✓

다변수(벡터) 경우로 확장

$z = f(x)$에서 $z, x \in \mathbb{R}^d$일 때:

$$\boxed{p_X(x) = p_Z(f(x)) \left| \det \frac{\partial f(x)}{\partial x} \right|}$$

여기서 $\frac{\partial f(x)}{\partial x}$는 **자코비안 행렬(Jacobian matrix)**이다:

$$J = \frac{\partial f}{\partial x} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_d} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_d}{\partial x_1} & \frac{\partial f_d}{\partial x_2} & \cdots & \frac{\partial f_d}{\partial x_d} \end{pmatrix}$$

$|\det J|$는 변환에 의한 **부피 변화율**을 나타낸다.

2D 예시

$f(x_1, x_2) = (2x_1, 3x_2)$이면:

$$J = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}, \quad \det J = 6$$

면적이 6배로 늘어나므로, 확률 밀도는 $\frac{1}{6}$로 줄어든다.

4.3 Log-Likelihood 계산

기본 식

$$p_X(x) = p_Z(f(x))\,|\det J_f(x)|$$

양변에 로그를 취하면:

$$\log p_X(x) = \log p_Z(f(x)) + \log|\det J_f(x)|$$

여기서:

- $\log p_Z(f(x))$: 변환된 값의 prior 확률 (표준 정규분포이므로 계산 가능)

$$\log p_Z(z) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\|z\|^2$$

- $\log|\det J_f(x)|$: 자코비안 행렬식의 로그 (변환의 부피 변화율)

Training Objective

Negative log-likelihood를 minimize:

$$\begin{aligned}\mathcal{L} &= -\log p_X(x) = -\log p_Z(f(x)) - \log|\det J_f(x)| \\ &= \frac{d}{2}\log(2\pi) + \frac{1}{2}\|f(x)\|^2 - \log|\det J_f(x)|\end{aligned}$$

VAE와의 비교

VAE:

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)\|p(z))$$

- 하한(lower bound)만 계산 가능
- 실제 likelihood는 알 수 없음

Normalizing Flow:

$$\log p(x) = \log p_Z(f(x)) + \log|\det J_f(x)|$$

- **정확한** likelihood 계산
- 모델 평가와 비교가 명확

4.4 합성 함수로의 확장

f를 단일 함수가 아닌 K개의 함수 합성으로 구성한다:

$$f = f_K \circ f_{K-1} \circ \cdots \circ f_1$$

각 f_k가 하나의 뉴럴넷 층에 해당한다.

```
x = z_0 → f_1 → z_1 → f_2 → z_2 → ... → f_K → z_K
```

자코비안의 chain rule

$$\frac{\partial z_K}{\partial x} = \frac{\partial f_K}{\partial z_{K-1}} \cdot \frac{\partial f_{K-1}}{\partial z_{K-2}} \cdots \frac{\partial f_1}{\partial x}$$

행렬식의 곱 성질

행렬식(determinant)은 곱에 대해 분리된다:

$$\det(AB) = \det(A) \cdot \det(B)$$

따라서:

$$\det \frac{\partial z_K}{\partial x} = \prod_{k=1}^{K} \det \frac{\partial f_k}{\partial z_{k-1}}$$

Log-likelihood

$$\log p_X(x) = \log p_Z(z_K) + \sum_{k=1}^{K} \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

여기서 $z_0 = x$이고 $z_K = f(x)$이다.

풀이 과정

출발점:

$$p_X(x) = p_Z(z_K) \cdot \prod_{k=1}^{K} \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

양변에 로그:

$$\log p_X(x) = \log p_Z(z_K) + \log \prod_{k=1}^{K} \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

로그 안의 곱을 합으로:

$$= \log p_Z(z_K) + \sum_{k=1}^{K} \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

> **핵심**: $\log p_Z(z_K)$는 표준 정규분포이므로 직접 계산 가능, 자코비안 항도 특수한 구조를 이용하면 효율적으로 계산 가능 → **정확한 log-likelihood 계산이 가능**

4.5 Affine Coupling Layer

문제: 역함수가 존재하는 뉴럴넷을 어떻게 만드는가?

일반적인 뉴럴넷은 비선형 활성화 함수 등으로 인해 역함수가 존재하지 않는다.

예: $y = \mathrm{ReLU}(Wx + b)$

- $x_1 = -1, x_2 = -2$가 모두 $y = 0$으로 매핑
- $y = 0$에서 역변환 시 x_1인지 x_2인지 알 수 없음

해결: Additive Coupling (NICE, 2014)

입력 x를 절반으로 분할한다: $x = [x_a, x_b]$

Forward:

$$y_a = x_a$$

$$y_b = x_b + m(x_a)$$

여기서 m은 임의의 뉴럴넷이다.

Inverse:

$$x_a = y_a$$

$$x_b = y_b - m(y_a) = y_b - m(x_a)$$

> $y_a = x_a$이므로 $m(y_a) = m(x_a)$가 성립하여 역변환이 가능하다.

왜 역함수가 존재하는가?

- x_a는 변환 없이 그대로 통과 → y_a에서 x_a 복원 가능
- x_a를 알면 $m(x_a)$를 계산 가능
- $y_b - m(x_a) = x_b$ 계산 가능

m이 아무리 복잡해도 (심지어 non-invertible해도) 전체 변환은 invertible하다!

Affine Coupling (Real NVP, 2016)

Additive coupling을 확장하여 **곱셈(scale)과 덧셈(translation)**을 모두 적용한다:

Forward:

$$y_a = x_a$$

$$y_b = x_b \odot \exp(s(x_a)) + t(x_a)$$

여기서 s (scale)과 t (translation)은 파라미터를 갖는 뉴럴넷이다.

Inverse 유도:

$y_b = x_b \odot \exp(s(x_a)) + t(x_a)$에서 x_b를 구하면:

1단계 — $t(x_a)$를 이항:

$$y_b - t(x_a) = x_b \odot \exp(s(x_a))$$

2단계 — $\exp(s(x_a))$로 나눔 (= $\exp(-s(x_a))$를 곱):

$$x_b = (y_b - t(x_a)) \odot \exp(-s(x_a))$$

$x_a = y_a$이므로:

$$\boxed{x_b = (y_b - t(y_a)) \odot \exp(-s(y_a))}$$

> s, t가 아무리 복잡한 뉴럴넷이더라도, 이 coupling 구조를 사용하면 항상 역변환이 가능하다.

왜 exp를 사용하는가?

$\exp(s(x_a))$를 사용하는 이유:

1. **항상 양수**: $\exp(s) > 0$ → scale factor가 0이 되지 않음 → 역변환 안정적
2. **수치 안정성**: $\log \exp(s) = s$ → 자코비안 계산 간단
3. **표현력**: s가 음수일 때 축소, 양수일 때 확대

만약 $s(x_a)$를 직접 사용하면:

- $s(x_a) = 0$일 때 정보 손실 (역변환 불가)
- 음수 scale은 부호 반전 (복잡성 증가)

4.6 자코비안 계산

Affine coupling의 자코비안을 구체적으로 계산한다.

자코비안 행렬 구성

$y = [y_a, y_b]$를 $x = [x_a, x_b]$에 대해 미분한 자코비안:

$$J = \frac{\partial y}{\partial x} = \begin{pmatrix} \frac{\partial y_a}{\partial x_a} & \frac{\partial y_a}{\partial x_b} \\ \frac{\partial y_b}{\partial x_a} & \frac{\partial y_b}{\partial x_b} \end{pmatrix}$$

각 블록을 계산하면:

$\frac{\partial y_a}{\partial x_a}$

$y_a = x_a$이므로:

$$\frac{\partial y_a}{\partial x_a} = I \quad (\text{항등 행렬})$$

$\frac{\partial y_a}{\partial x_b}$

$y_a = x_a$는 x_b에 의존하지 않으므로:

$$\frac{\partial y_a}{\partial x_b} = 0$$

$\frac{\partial y_b}{\partial x_a}$

$y_b = x_b \odot \exp(s(x_a)) + t(x_a)$에서 x_a에 대한 편미분은 복잡하지만, 이 위치는 자코비안의 **하삼각 부분**이다.

$\frac{\partial y_b}{\partial x_b}$

$y_b = x_b \odot \exp(s(x_a)) + t(x_a)$에서 x_b에 대해 미분하면:

$$\frac{\partial y_b}{\partial x_b} = \text{diag}(\exp(s(x_a)))$$

여기서 diag는 대각 행렬을 의미한다. x_b의 각 원소가 독립적으로 $\exp(s(x_a))$의 해당 원소와 곱해지므로 대각 형태가 된다.

자코비안 행렬의 형태

$$J = \begin{pmatrix} I & 0 \\ \frac{\partial y_b}{\partial x_a} & \text{diag}(\exp(s(x_a))) \end{pmatrix}$$

이것은 **하삼각 행렬(lower triangular matrix)**이다.

행렬식 계산

하삼각 행렬의 행렬식은 **대각 원소의 곱**이다:

$$\det J = \det(I) \cdot \det(\text{diag}(\exp(s(x_a)))) = 1 \cdot \prod_j \exp(s_j(x_a))$$

$$= \exp\left(\sum_j s_j(x_a)\right)$$

로그 행렬식

$$\log|\det J| = \log\exp\left(\sum_j s_j(x_a)\right) = \sum_j s_j(x_a)$$

> **핵심 결과**: 로그 행렬식이 **단순한 합산**으로 계산된다. 행렬식 계산의 $O(d^3)$ 복잡도가 $O(d)$로 줄어든다.

왜 중요한가?

일반 행렬 $d \times d$의 행렬식 계산:

- LU 분해 사용: $O(d^3)$ 시간
- 이미지 $256 \times 256 \times 3$: $d = 196,608$ → 계산 불가능

Affine coupling:

- 단순 합산: $O(d)$ 시간
- 실시간 계산 가능

4.7 교대 분할 (Alternating Pattern)

Affine coupling에서 x_a는 변환되지 않고 그대로 통과한다. 이를 보완하기 위해:

- **홀수 층**: $[x_a, x_b] \to [x_a, \text{transform}(x_b)]$
- **짝수 층**: $[x_a, x_b] \to [\text{transform}(x_a), x_b]$

이렇게 교대하면 모든 차원이 변환의 영향을 받게 된다.

구체적 예시

3개 층 (각 층은 affine coupling):

```
Layer 1: [x_1, x_2] → [x_1, f_1(x_1, x_2)]          # x_2 변환
Layer 2: [x_1, x_2] → [f_2(x_1, x_2), x_2]          # x_1 변환
Layer 3: [x_1, x_2] → [x_1, f_3(x_1, x_2)]          # x_2 변환
```

최종적으로 x_1, x_2 모두 변환의 영향을 받음.

Multi-scale Architecture

Real NVP는 다음과 같은 구조를 사용:

1. **Checkerboard pattern**: 체스판 형태로 픽셀 분할
2. **Channel-wise pattern**: 채널 단위로 분할
3. **Squeeze operation**: 해상도를 절반으로, 채널을 4배로

이를 통해 효율적으로 전체 이미지를 변환한다.

4.8 전체 학습 및 생성 파이프라인

훈련

```
x → f₁ → z₁ → f₂ → z₂ → ... → fₖ → zₖ (≈ N(0,I))
```

목적 함수:

$$\mathcal{L} = -\log p_Z(z_K) - \sum_{k=1}^{K} \log|\det J_{f_k}|$$

$$= -\log \mathcal{N}(z_K; 0, I) - \sum_{k=1}^{K} \sum_j s_j^{(k)}$$

표준 정규분포의 로그확률:

$$\log \mathcal{N}(z; 0, I) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\|z\|^2$$

따라서:

$$\mathcal{L} = \frac{d}{2}\log(2\pi) + \frac{1}{2}\|z_K\|^2 - \sum_{k=1}^{K} \sum_j s_j^{(k)}$$

훈련 알고리즘

```
for each batch (x_1, ..., x_B):
    z = x
    log_det_sum = 0

    for k = 1 to K:
        z = f_k(z)
        log_det_sum += log|det J_{f_k}|

    loss = 0.5 * ||z||^2 - log_det_sum
    loss.backward()
    optimizer.step()
```

생성

```
zₖ ~ N(0,I) → f⁻¹ₖ → ... → f⁻¹₂ → f⁻¹₁ → x
```

역변환을 순차적으로 적용하여 이미지를 생성한다.

생성 알고리즘

```
z_K ~ N(0, I)

for k = K down to 1:
    z_{k-1} = f_k^{-1}(z_k)

x = z_0
```

Affine coupling은 역함수가 명시적이므로 생성이 매우 빠르다.

4.9 VAE와의 비교

항목	VAE	Normalizing Flow
목적 함수	ELBO (하한)	**정확한 log-likelihood**
Unknown term	$D_{KL}(q_\phi \| p_\theta) \geq 0$	**없음**
네트워크	인코더 + 디코더 (별도)	**하나** (invertible)
잠재 공간 차원	입력보다 낮음	입력과 **동일**
이론적 완벽성	근사	**정확**
구현 제약	없음	invertible 구조 필요
훈련 속도	빠름	보통 (자코비안 계산)
생성 속도	빠름	**매우 빠름** (1회 통과)
잠재 공간 보간	부드러움	덜 부드러움

실제 성능 비교 (CIFAR-10)

모델	Bits per dim	파라미터 수
VAE	~4.5	~10M
PixelCNN	3.03	~5M
Real NVP	3.49	~116M
Glow	3.35	~44M

4.10 Normalizing Flow → Flow Matching으로의 발전

Normalizing Flow의 K개 이산 층을 **연속적인 시간 변수**로 대체하면:

Continuous Normalizing Flow (CNF)

이산: $f = f_K \circ f_{K-1} \circ \cdots \circ f_1$

연속: $\frac{dz(t)}{dt} = v_\theta(z(t), t), \quad t \in [0, 1]$

이것이 **Neural ODE**이다.

Flow Matching (2022)

Neural ODE의 효율적 훈련 방법:

- Continuous normalizing flow를 simulation-free로 학습
- ODE solver 없이 직접 velocity field 학습
- 현재 가장 성능이 좋고 빠른 생성 모델 중 하나

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x,z} \| v_\theta(z_t, t) - (x - z) \|^2$$

발전 경로

```
Normalizing Flow (2014-2017)
    ↓
Continuous Normalizing Flow / Neural ODE (2018)
    ↓
FFJORD (2018)
    ↓
Flow Matching (2022)
    ↓
Modern generative models (Stable Diffusion 3, etc.)
```

이것이 Normalizing Flow를 학습하는 것이 중요한 이유이다 — **Flow Matching의 이론적 기반**이 된다.

4.11 PyTorch 구현

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class AffineCoupling(nn.Module):
    """
    Affine Coupling Layer for Normalizing Flow

    Args:
        in_channels: Number of input channels
        hidden_channels: Number of hidden channels in the NN
        mask_type: 'channel_wise' or 'checkerboard'
    """
    def __init__(self, in_channels, hidden_channels, mask_type='char
        super(AffineCoupling, self).__init__()

        self.mask_type = mask_type

        # NN to compute scale (s) and translation (t)
        # Input: half of channels, Output: 2x half of channels (for
        self.net = nn.Sequential(
            nn.Conv2d(in_channels // 2, hidden_channels, kernel_size
            nn.ReLU(inplace=True),
            nn.Conv2d(hidden_channels, hidden_channels, kernel_size=
            nn.ReLU(inplace=True),
            nn.Conv2d(hidden_channels, in_channels, kernel_size=3, p
        )

        # Initialize last layer to zero (identity transform initiall
        self.net[-1].weight.data.zero_()
        self.net[-1].bias.data.zero_()

    def forward(self, x, reverse=False):
        """
        Forward or inverse transformation

        Args:
            x: Input tensor of shape (B, C, H, W)
            reverse: If True, perform inverse transformation

        Returns:
            y: Transformed tensor
            log_det: Log determinant of Jacobian
        """
        # Split input
        x_a, x_b = x.chunk(2, dim=1)

        # Compute scale and translation from x_a
        st = self.net(x_a)
        s, t = st.chunk(2, dim=1)

        # Numerically stable scale
```

```python
        s = torch.tanh(s)  # Bound scale to [-1, 1]

        if not reverse:
            # Forward: y_b = x_b * exp(s) + t
            y_b = x_b * torch.exp(s) + t
            y = torch.cat([x_a, y_b], dim=1)

            # Log determinant: sum of log scale
            log_det = torch.sum(s, dim=[1, 2, 3])
        else:
            # Inverse: x_b = (y_b - t) * exp(-s)
            x_b = (x_b - t) * torch.exp(-s)
            y = torch.cat([x_a, x_b], dim=1)

            # Log determinant: negative of forward
            log_det = -torch.sum(s, dim=[1, 2, 3])

        return y, log_det


class Squeeze(nn.Module):
    """
    Squeeze operation: (B, C, H, W) -> (B, 4C, H/2, W/2)
    Rearranges spatial dimensions into channels
    """
    def forward(self, x, reverse=False):
        B, C, H, W = x.shape

        if not reverse:
            # Squeeze: (B, C, H, W) -> (B, 4C, H/2, W/2)
            x = x.view(B, C, H // 2, 2, W // 2, 2)
            x = x.permute(0, 1, 3, 5, 2, 4).contiguous()
            x = x.view(B, C * 4, H // 2, W // 2)
        else:
            # Unsqueeze: (B, 4C, H/2, W/2) -> (B, C, H, W)
            x = x.view(B, C // 4, 2, 2, H, W)
            x = x.permute(0, 1, 4, 2, 5, 3).contiguous()
            x = x.view(B, C // 4, H * 2, W * 2)

        return x


class RealNVP(nn.Module):
    """
    Real NVP: Normalizing Flow model

    Args:
        in_channels: Number of input channels (3 for RGB)
        hidden_channels: Number of hidden channels in coupling layer
        num_scales: Number of multi-scale levels
        num_layers_per_scale: Number of coupling layers per scale
    """
```

```python
def __init__(self, in_channels=3, hidden_channels=64, num_scales
    super(RealNVP, self).__init__()

    self.in_channels = in_channels
    self.num_scales = num_scales

    self.flows = nn.ModuleList()
    self.squeezes = nn.ModuleList()

    current_channels = in_channels

    for scale in range(num_scales):
        # Squeeze operation (except for first scale)
        if scale > 0:
            squeeze = Squeeze()
            self.squeezes.append(squeeze)
            current_channels *= 4

        # Coupling layers for this scale
        scale_flows = nn.ModuleList()
        for _ in range(num_layers_per_scale):
            scale_flows.append(
                AffineCoupling(current_channels, hidden_channels
            )
        self.flows.append(scale_flows)

def forward(self, x, reverse=False):
    """
    Forward or inverse transformation

    Args:
        x: Input tensor
        reverse: If True, perform generation (inverse)

    Returns:
        z: Latent representation (if forward) or generated image
        log_det: Total log determinant
    """
    log_det_total = 0

    if not reverse:
        # Forward: x -> z
        z = x
        squeeze_idx = 0

        for scale, scale_flows in enumerate(self.flows):
            # Squeeze (except first scale)
            if scale > 0:
                z = self.squeezes[squeeze_idx](z, reverse=False
                squeeze_idx += 1
```

```
            # Apply coupling layers
            for flow in scale_flows:
                z, log_det = flow(z, reverse=False)
                log_det_total += log_det

        return z, log_det_total
    else:
        # Inverse: z -> x
        x = x
        squeeze_idx = len(self.squeezes) - 1

        for scale in range(self.num_scales - 1, -1, -1):
            scale_flows = self.flows[scale]

            # Apply coupling layers in reverse
            for flow in reversed(scale_flows):
                x, log_det = flow(x, reverse=True)
                log_det_total += log_det

            # Unsqueeze (except first scale)
            if scale > 0:
                x = self.squeezes[squeeze_idx](x, reverse=True)
                squeeze_idx -= 1

        return x, log_det_total

def sample(self, n_samples, img_shape=(3, 32, 32), device='cuda
    """
    Generate samples from the model

    Args:
        n_samples: Number of samples to generate
        img_shape: Shape of generated images (C, H, W)
        device: Device to generate on
        temperature: Sampling temperature (default 1.0)

    Returns:
        samples: Generated images
    """
    self.eval()

    # Compute latent shape after all squeezes
    C, H, W = img_shape
    for _ in range(self.num_scales - 1):
        C *= 4
        H //= 2
        W //= 2

    # Sample from Gaussian
    with torch.no_grad():
        z = torch.randn(n_samples, C, H, W, device=device) * ter
        samples, _ = self.forward(z, reverse=True)
```

```
        return samples


# Training function
def train_realnvp(model, train_loader, optimizer, device, num_epochs
    """
    Train Real NVP model

    Args:
        model: RealNVP model
        train_loader: DataLoader for training data
        optimizer: Optimizer
        device: Device to train on
        num_epochs: Number of training epochs
    """
    model.train()

    for epoch in range(num_epochs):
        total_loss = 0
        total_bpd = 0  # bits per dimension

        for batch_idx, (data, _) in enumerate(train_loader):
            data = data.to(device)
            B, C, H, W = data.shape
            n_pixels = C * H * W

            optimizer.zero_grad()

            # Forward pass
            z, log_det = model(data, reverse=False)

            # Compute log probability
            log_pz = -0.5 * torch.sum(z**2, dim=[1, 2, 3]) - 0.5 * 
            log_px = log_pz + log_det

            # Negative log-likelihood loss
            loss = -torch.mean(log_px)

            # Bits per dimension
            bpd = loss / (n_pixels * torch.log(torch.tensor(2.0)))

            # Backward pass
            loss.backward()

            # Gradient clipping
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_

            optimizer.step()

            total_loss += loss.item()
            total_bpd += bpd.item()
```

```
        total_bpd = bpd.item()

    avg_loss = total_loss / len(train_loader)
    avg_bpd = total_bpd / len(train_loader)

    print(f"Epoch {epoch+1}/{num_epochs}")
    print(f"  Loss: {avg_loss:.4f}")
    print(f"  Bits/dim: {avg_bpd:.4f}")


# Example usage
if __name__ == "__main__":
    # Hyperparameters
    device = 'cuda' if torch.cuda.is_available() else 'cpu'

    # Model
    model = RealNVP(
        in_channels=3,
        hidden_channels=64,
        num_scales=2,
        num_layers_per_scale=8
    ).to(device)

    # Optimizer
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

    # Training (assuming train_loader is defined)
    # train_realnvp(model, train_loader, optimizer, device, num_epoc

    # Generate samples
    # samples = model.sample(n_samples=16, img_shape=(3, 32, 32), de
```

주요 하이퍼파라미터

파라미터	권장 값	설명
hidden_channels	64-512	Coupling layer의 hidden channels
num_scales	2-4	Multi-scale 레벨 수
num_layers_per_scale	4-12	각 scale의 coupling layer 수
learning_rate	1e-4 to 5e-4	Adam optimizer
batch_size	32-128	GPU 메모리에 따라 조정
temperature	0.7-1.0	샘플링 온도 (낮을수록 선명)

성능 최적화 팁

1. **Batch Normalization**: Coupling layer에 ActNorm 추가
2. **1×1 Convolution**: Invertible 1×1 conv로 채널 mixing
3. **Gradient Clipping**: 안정적 훈련을 위해 gradient clipping

4. **Warmup**: Learning rate warmup으로 초기 안정성 향상

---

5. 전체 요약: 세 모델의 비교

	PixelRNN/CNN	VQ-VAE	Normalizing Flow
접근 방식	Autoregressive	Discrete Latent	Invertible Network
확률 분포	$\prod p(x_i \mid x_{<i})$	Codebook + Prior	Change of Variables
생성 속도	느림 (순차)	빠름 (디코더 1회)	빠름 (역변환 1회)
훈련 속도	PixelCNN 빠름	보통	보통 (자코비안 계산)
이론적 정확성	Exact likelihood	2-stage	**Exact likelihood**
차원	동일	압축 (32×32)	동일
네트워크	Masked Conv	Enc + Dec + Codebook	Invertible layers
주요 기여	WaveNet 등 오디오	이산 표현 학습	Flow Matching의 기반
현대적 응용	트랜스포머 prior	DALL-E, VQ-GAN	Stable Diffusion 3

5.1 각 모델의 강점과 약점

PixelRNN/CNN

강점:

- Exact likelihood 계산 가능
- 이론적으로 완벽 (어떤 근사도 없음)
- 오디오에서 큰 성공 (WaveNet)

약점:

- 생성 속도 매우 느림
- 이미지 품질이 GAN/Diffusion보다 낮음
- Long-range dependency 학습 어려움

VQ-VAE

강점:

- Posterior collapse 없음
- 이산 표현으로 트랜스포머와 호환

- 압축 효율 높음

약점:

- 2-stage 훈련 필요
- 코드북 학습 불안정할 수 있음
- Codebook collapse 문제

Normalizing Flow

강점:

- Exact likelihood 계산
- 역변환이 명시적 (빠른 생성)
- 이론적으로 우아함

약점:

- Invertible 구조 제약
- 잠재 공간 차원 = 입력 차원 (압축 안됨)
- 큰 모델 필요 (파라미터 수 많음)

5.2 현대 생성 모델과의 연결

Diffusion Models

- Normalizing Flow + Stochastic process
- DDPM, DDIM 등은 discrete-time flow로 볼 수 있음

Flow Matching

- Continuous Normalizing Flow의 효율적 학습
- Stable Diffusion 3, Flux 등에서 사용

Transformer + VQ

- DALL-E: dVAE (VQ-VAE 변형) + GPT
- MaskGIT: VQ-GAN + Masked transformer
- ImageGPT: PixelCNN 아이디어 + GPT

---

6. 참고문헌

PixelRNN/PixelCNN

1. **Pixel Recurrent Neural Networks** van den Oord, A., Kalchbrenner, N., & Kavukcuoglu, K. (2016) ICML 2016 https://arxiv.org/abs/1601.06759

2. **Conditional Image Generation with PixelCNN Decoders** van den Oord, A., et al. (2016) NeurIPS 2016 https://arxiv.org/abs/1606.05328

3. **WaveNet: A Generative Model for Raw Audio** van den Oord, A., et al. (2016) https://arxiv.org/abs/1609.03499

VQ-VAE

4. **Neural Discrete Representation Learning** van den Oord, A., Vinyals, O., & Kavukcuoglu, K. (2017) NeurIPS 2017 https://arxiv.org/abs/1711.00937

5. **Generating Diverse High-Fidelity Images with VQ-VAE-2** Razavi, A., van den Oord, A., & Vinyals, O. (2019) NeurIPS 2019 https://arxiv.org/abs/1906.00446

6. **Zero-Shot Text-to-Image Generation (DALL-E)** Ramesh, A., et al. (2021) ICML 2021 https://arxiv.org/abs/2102.12092

Normalizing Flow

7. **NICE: Non-linear Independent Components Estimation** Dinh, L., Krueger, D., & Bengio, Y. (2014) ICLR 2015 Workshop https://arxiv.org/abs/1410.8516

8. **Density estimation using Real NVP** Dinh, L., Sohl-Dickstein, J., & Bengio, S. (2016) ICLR 2017 https://arxiv.org/abs/1605.08803

9. **Glow: Generative Flow using Invertible 1x1 Convolutions** Kingma, D. P., & Dhariwal, P. (2018) NeurIPS 2018 https://arxiv.org/abs/1807.03039

10. **Neural Ordinary Differential Equations** Chen, R. T. Q., et al. (2018) NeurIPS 2018 https://arxiv.org/abs/1806.07366

11. **Flow Matching for Generative Modeling** Lipman, Y., et al. (2022) ICLR 2023 https://arxiv.org/abs/2210.02747

추가 자료

12. **Tutorial on Normalizing Flows** Kobyzev, I., Prince, S., & Brubaker, M. (2020) IEEE Transactions on Pattern Analysis and Machine Intelligence https://arxiv.org/abs/1908.09257

13. **VQ-GAN: Taming Transformers for High-Resolution Image Synthesis** Esser, P., Rombach, R., & Ommer, B. (2021) CVPR 2021 https://arxiv.org/abs/2012.09841

다음 강의 예고: Lecture 3에서는 **GAN (Generative Adversarial Networks)**을 다룹니다. 적대적 학습의 원리, 다양한 GAN 변형 모델, 그리고 실전 훈련 기법을 학습합니다.